

blockst Version 0.4.0

Render Scratch, Blockly, MakeCode and Open Roberta blocks for educational documents.

blockst renders programming blocks directly in Typst documents — Scratch, Blockly with a jwinf profile, MakeCode for the micro:bit and the Calliope mini, and Open Roberta NEPO — for worksheets, tutorials and teaching material. Block code is written as plain text and rendered by a bundled WASM plugin, Scratch in 26 languages including right-to-left scripts, with a turtle-graphics execution engine and helpers for importing real `.sb3` project files.

Inhalt

1	Introduction	3
1.1	About blockst	3
1.2	Quick start	3
1.3	Package information	3
2	Core Rendering API	5
2.1	scratch() — Render Scratch Blocks	5
2.1.1	Signature	5
2.1.2	Parameters	5
2.1.3	Supported block syntax	6
2.2	set-blockst() — Global Defaults	6
2.2.1	Themes	6
2.2.2	Fonts	7
2.2.3	Line numbers	7
2.2.4	Inset scale	7
2.3	blockst() — Group Override Container	7
3	Languages	9
3.1	Right-to-left languages	9
4	Blockly and jwinf	11
5	MakeCode	12
6	Labels and Line Numbers	13
6.1	scratch-labels() — Extract Labels	13
6.2	blockst-labels() — Query Labels	13
6.3	blockst-register-labels() — Pre-register Labels	13
7	@category — Quick Color Defaults	15
8	Parsing API	16
8.1	scratch-parse() — Parse to AST	16
9	Markdown Code Blocks with raw-scratch	17
10	Theme and Styling Examples	18
11	Turtle Graphics / Executable Scratch	19
11.1	scratch-run Module	19
11.1.1	scratch-run.stage() — Stage Canvas	19
11.1.2	scratch-run.grid() — Coordinate Grid	19
11.1.3	set-scratch-run() — Run Global Defaults	19
11.1.4	Supported Execution Commands	20
11.1.5	Complete Example	21
12	SB3 Import API	24
12.1	Basic Workflow	24
12.2	render-sb3-scripts()	24
12.3	render-sb3-variables()	25
12.4	render-sb3-lists()	25
12.5	Standalone Monitors	26
12.5.1	list-monitor()	26

12.5.2	variable-monitor()	26
12.6	screen-preview()	26
12.7	Image Helpers	26
12.8	Catalog Helpers	27
13	Catalog	28
13.1	Motion	28
13.2	Looks	29
13.3	Sound	31
13.4	Pen	32
13.5	Variables	33
13.6	Lists	33
13.7	Events	34
13.8	Control	35
13.9	Sensing	37
13.10	Operators	38
14	Contributing	40
15	API Reference	41
15.1	blockly	41
15.2	blockly-parse	42
15.3	blockst	42
15.4	blockst-labels	43
15.5	blockst-register-labels	43
15.6	makecode	43
15.7	makecode-parse	44
15.8	raw-blockly	44
15.9	raw-makecode	44
15.10	raw-scratch	44
15.11	scratch	45
15.12	scratch-execute	45
15.13	scratch-labels	45
15.14	scratch-parse	46
15.15	set-blockst	46

1 Introduction

1.1 About blockst

blockst renders programming blocks directly in Typst documents — Scratch, Blockly (with a profile for jwinf.de), MakeCode (micro:bit and Calliope mini) and Open Roberta NEPO, each drawn the way its editor draws it. It is designed for worksheets, tutorials, teaching material, and visual programming explanations — anything where block code needs to appear in print or online documentation.

The current renderer uses a text-to-WASM pipeline: Typst passes Scratch text to a bundled WASM plugin, the plugin parses and renders SVG, and Typst embeds the SVG output in the document.

CORE DESIGN

- Fully text-based: write Scratch blocks as plain text, get rendered blocks.
- 26 languages supported via built-in WASM locale data, including right-to-left scripts.
- *Localized* block rendering follows official Scratch translations.
- *Blockly* and *MakeCode* blocks from the same notation: `blockly()` and `makecode()`, with the editors' own block texts.
- *Turtle graphics* execution engine for demonstrating program flow visually.
- *SB3 import* helpers for reading real Scratch project files.

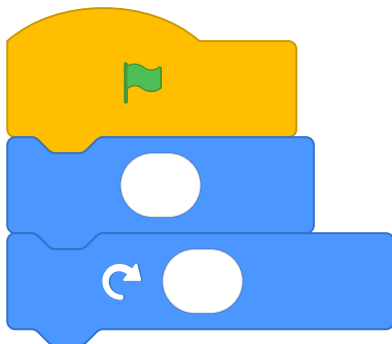
BREAKING CHANGE SINCE 0.2.0

The old pre-0.2.0 native-Typst renderer syntax is **removed**. From 0.2.0 onward, blockst uses only the text-to-WASM pipeline. Documents that still rely on the previous syntax must be migrated.

1.2 Quick start

```
#import "@preview/blockst:0.4.0": blockst, scratch, raw-scratch, sb3

#scratch("
when green flag clicked
move (10) steps
turn cw (15) degrees
")
```



1.3 Package information

- **Version:** 0.4.0
- **License:** MIT

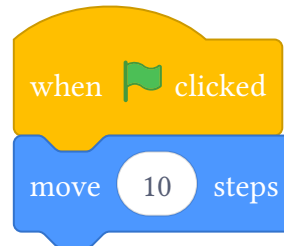
- **Repository:** github.com/Loewe1000/blockst
- **Compiler requirement:** Typst 0.15.0+
- **Font requirement:** Designed for Helvetica Neue (Scratch look). On Linux/Windows install a compatible font (e.g. Nimbus Sans) or override via [set-blockst](#). MakeCode blocks use a monospace face (Menlo, Consolas or DejaVu Sans Mono, whichever is installed).

2 Core Rendering API

2.1 scratch() — Render Scratch Blocks

The primary function for rendering Scratch blocks. It parses Scratch-like text and produces visual blocks.

```
#scratch("when green flag clicked\nmove (10)\nsteps")
```



2.1.1 Signature

```
#scratch(  
  text,  
  language: "en",  
  theme: auto,  
  scale: auto,  
  line-numbers: auto,  
  line-number-start: auto,  
  line-number-gutter: auto,  
  inset-scale: auto,  
)
```

2.1.2 Parameters

Name	Default	Description
text	– (required)	Scratch block text. Supports blocks, reporters, booleans, inputs, dropdowns, and nested control structures.
language	"en"	Locale for block text. See Languages for available options.
theme	auto	Visual theme: "normal", "high-contrast", "print", "grayscale", or auto (global default).
scale	auto	Overall size multiplier (e.g. 80%, 0.8, 50%).
font	auto	Font for block labels. Set it here rather than with <code>#set text(font: ...)</code> : the renderer measures each label and writes the font into the SVG, so a document-level <code>set text</code> changes what is drawn but not what was measured, and the labels overlap.
line-numbers	auto	Show line numbers (true or false).
line-number-start	auto	Starting line number (integer, default 1).
line-number-gutter	auto	Width of line number gutter (pt, default 24).
inset-scale	auto	Proportional block geometry scaling. 100% = default, 60% = thin blocks, 125% = thick blocks. Does not affect text size.

2.1.3 Supported block syntax

The text parser supports the full Scratch 3 block vocabulary in all 26 locales. Key syntax patterns:

Pattern	Example	Description
Stack block	<code>move (10) steps</code>	Standard command block
Hat block	<code>when green flag clicked</code>	Event trigger
Reporter	<code>(x position)</code>	Value reporter (round ends)
Boolean	<code><mouse down?></code>	Predicate (pointy ends)
Input: number	<code>(10)</code>	Numeric input field
Input: string	<code>[hello]</code>	String input field
Input: dropdown	<code>[random position v]</code>	Dropdown selector
C-block	<code>repeat (4)\n...\nend</code>	Block with body
C-block + else	<code>if <...> then\n...\nelse\n...\nend</code>	Conditional with else
Line labels	<code>move (10) steps #step</code>	Named label (see Labels)
Category prefix	<code>@motion free text</code>	Force category color (see category)
Category prefix (render)	<code>#text("@category") + text</code>	Matches real block when text fits

2.2 set-blockst() — Global Defaults

Sets global rendering options for all subsequent `scratch()` and SB3 render calls.

```
#set-blockst(
  theme: none,
  scale: none,
  stroke-width: none,
  font: none,
  line-numbers: none,
  line-number-start: none,
  line-number-gutter: none,
  inset-scale: none,
)
```

All parameters are optional. Only provided values override the current defaults.

2.2.1 Themes

Theme	Description
"normal"	Default — full color Scratch style.
"high-contrast"	Lighter, high-contrast variant for accessibility.
"print"	Every block white with a black outline. The lightest on ink, but all categories look alike.
"grayscale"	One distinct grey per category, so a control block still reads differently from an operator on a monochrome page.

```
#set-blockst(theme: "print", scale: 70%)
#scratch("when green flag clicked\nmove (10) steps")
```

2.2.2 Fonts

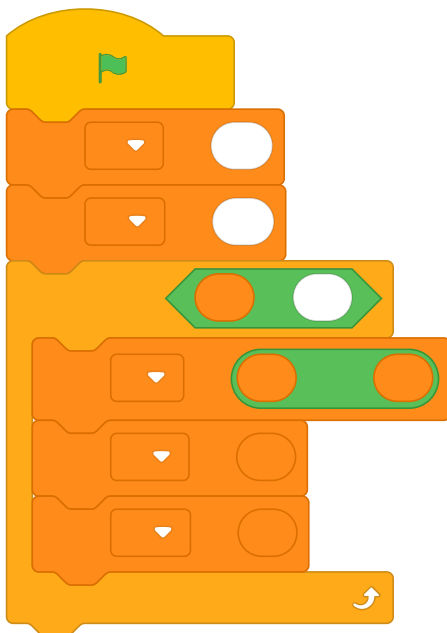
```
#set-blockst(font: "Nimbus Sans")
```

The default font is Helvetica Neue. On systems without it, set an alternative.

2.2.3 Line numbers

```
#set-blockst(line-numbers: true, line-number-start: 1, line-number-gutter: 24)
```

Euclidean Algorithm (gcd label walkthrough)



Explanation of labeled lines

- **Line 4:** Loop condition $b \neq 0$ controls termination.
- **Line 5:** Core rule: $r = a \bmod b$.
- **Line 7:** State update that advances to the next pair.

2.2.4 Inset scale

Controls the visual thickness/slimness of blocks without changing text size.

```
#set-blockst(inset-scale: 60%) // compact blocks
#set-blockst(inset-scale: 125%) // generous blocks
```

2.3 blockst() — Group Override Container

Optional wrapper for grouped blocks that differ from global settings.

```
#blockst(
  theme: auto,
  scale: auto,
  line-numbers: auto,
  line-number-start: auto,
  line-number-gutter: auto,
  inset-scale: auto,
  spacing: 1.5em,
  body,
)
```

All parameters match `scratch()` but apply only within the body. Use when a specific group of blocks needs different scaling or theme.

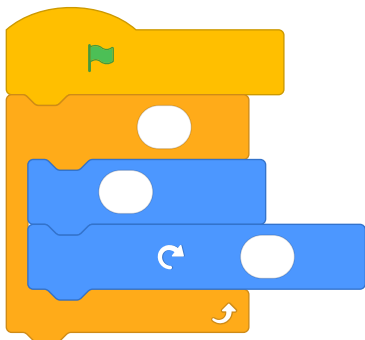
```
#blockst(theme: "high-contrast") [  
  #scratch("when green flag clicked\nmove (10) steps")  
]
```

3 Languages

blockst supports **26 languages** via built-in WASM locale data. Set via `scratch(..., language: "de")` or `set-blockst(...)`.

Code	Language	Code	Language
"en"	English	"de"	German
"fr"	French	"es"	Spanish
"it"	Italian	"nl"	Dutch
"pt"	Portuguese	"pl"	Polish
"ru"	Russian	"ja"	Japanese
"ca"	Catalan	"cs"	Czech
"cy"	Welsh	"el"	Greek
"fa"	Persian	"gd"	Scottish Gaelic
"he"	Hebrew	"hi"	Hindi
"hr"	Croatian	"hu"	Hungarian
"id"	Indonesian	"nb"	Norwegian (Bokmål)
"ro"	Romanian	"sl"	Slovenian
"tr"	Turkish	"ar"	Arabic

```
#set-blockst(scale: 67.5%)
#scratch("
Wenn die grüne Flagge angeklickt
wiederhole (4) mal
  gehe (30) er Schritt
  drehe dich nach rechts um (90) Grad
end
", language: "de")
```



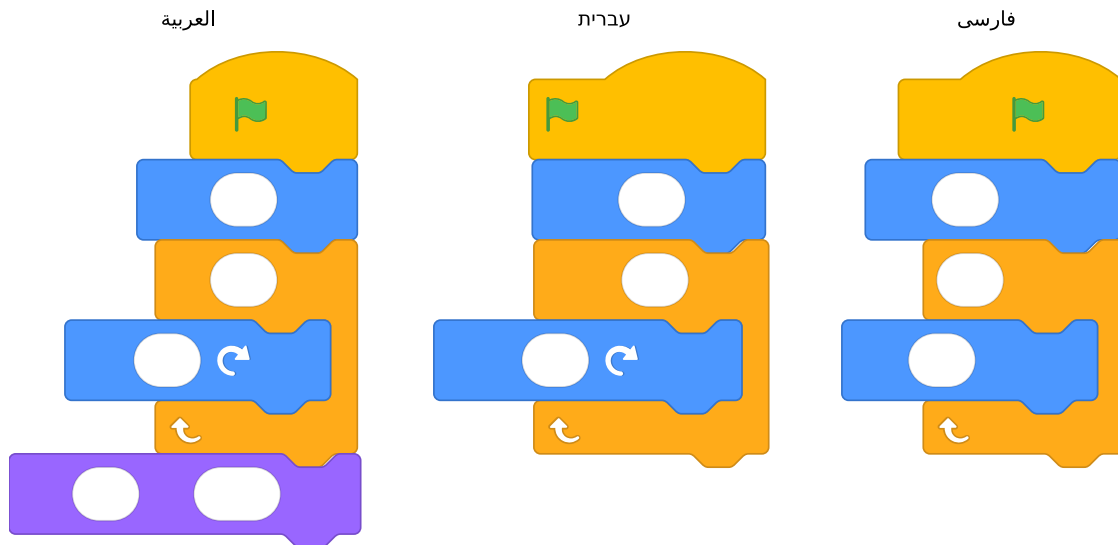
Block text must match the chosen locale's vocabulary — the parser matches against official Scratch translations.

3.1 Right-to-left languages

Arabic ("ar"), Hebrew ("he") and Persian ("fa") render right-to-left. Nothing has to be switched on: each locale declares its own direction, and the renderer mirrors the layout — the notch, the hat dome, the

C-block mouth, the loop arrow, the pen badge, the `define` hat and the order of the labels all move to the reading edge.

```
#scratch("
عند نقر @greenFlag
تحرك (10) خطوة
كرّر (4) مرة
درجة (90) @turnRight استدر
نهاية
", language: "ar")
```



WHAT IS MIRRORED, AND WHAT IS NOT

Layout is mirrored; meaning is not. The loop arrow points back to the top of the loop, which is a claim about the drawing, so it flips with the drawing. `@turnRight` and `@turnLeft` are never mirrored — their direction is what the sprite is being told to do, and a mirrored `@turnRight` would tell the reader to turn the other way.

Arabic short vowels are optional and their order is not canonical, so blocks match whether or not you type the harakat: `كرّر` and `كرّر` both find the repeat block.

4 Blockly and jwinf

`blockly()` renders Blockly blocks from the notation `scratch()` uses, drawn with Blockly's shapes. A profile chooses look and vocabulary: `"blockly"` (the default, today's flat look), `"blockly-klassisch"` (before 2019), `"jwinf"` — the Jugendwettbewerb Informatik's classic geometry, the palette of its robot training tasks, and its robot and turtle blocks — or `"jwinf-turtle"`, the same with the colours of the Freie Turtle-Umgebung.

The standard blocks come in 24 languages, from Blockly's own message files: `language: "de"` (the default), `"en"`, `"fr"` (`répéter (10) fois`), `"ja"` (`(10) 回繰り返す`), and `ar`, `ca`, `cs`, `el`, `es`, `fa`, `he`, `hi`, `hr`, `hu`, `id`, `it`, `nb`, `nl`, `pl`, `pt`, `ro`, `ru`, `sl`, `tr`. The jwinf world blocks exist in German only. Each language closes a C-block with the marker its Scratch locale uses (`ende`, `end`, `fin`, ...) and opens the else branch with Blockly's word (`sonst`, `else`, `sinon`, ...); `ende`, `end` and `else` work in every language.

```
#blockly("
  Roboter-Programm
  wiederhole (4) mal:
    gehe nach rechts
    falls <auf Kiste>
      hebe Marmor auf ::aktionen
    ende
  ende
", profile: "jwinf")
```

On jwinf a block's text changes from task to task, so unknown labels are drawn as written and take their category from a `::kategorie` suffix. Known blocks — loops, conditions, variables, the world commands — need none. `ende` closes a C-block, `sonst` opens its else branch, and `<...>` equals `(...)`: Blockly has no hexagonal boolean.

`raw-blockly()` renders ````blockly` and ````jwinf` fences; `blockly-parse()` returns the AST. `set-blockst(profile: ...)` sets the default profile for `blockly()` only. Themes apply as for Scratch, with `grayscale` derived from the profile's palette.

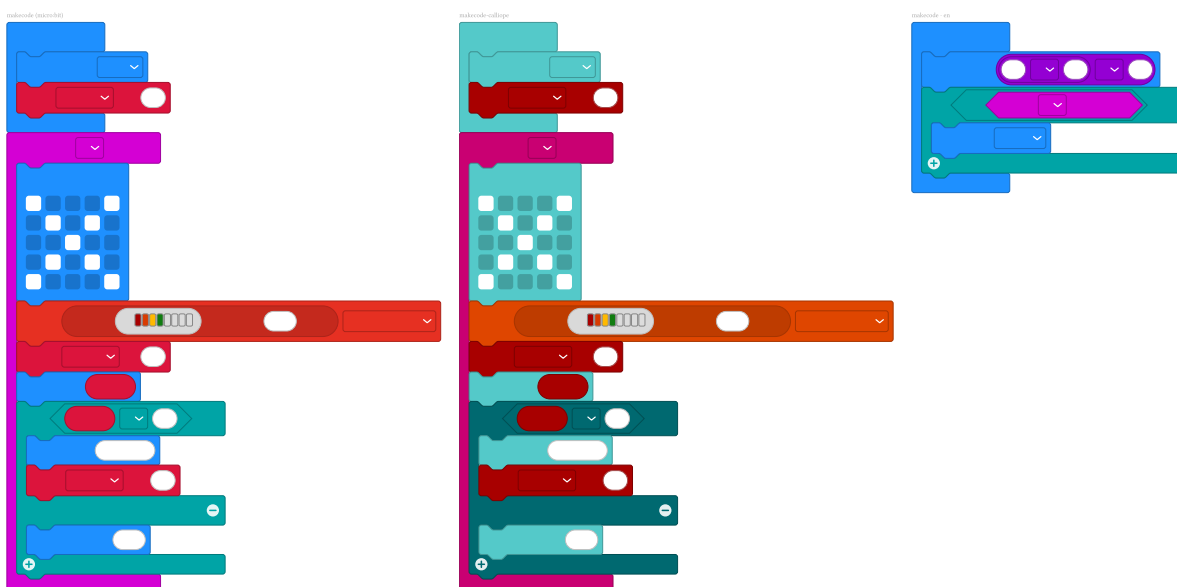
jwinf colours its categories per task family. Where a task deviates from both profiles, `colors` lays the document's own category colours over the palette — `set-blockst(colors: (logik: "#73cc47"))` for the document, or `colors: on` on `blockly()`, `scratch()` and `blockst()` for a single block or group. A hex string or a Typst colour per category; the derived shades (`bevel`, `stroke`, `high-contrast`, `grayscale`) follow the new fill.

5 MakeCode

`makecode()` renders the blocks of the MakeCode editors for the micro:bit (profile: "makecode", the default) and the Calliope mini ("makecode-calliope"), from the notation `scratch()` uses. The look is Blockly's `zelos` renderer as the editors run it — pills, hexagons, white literal pills, MakeCode's monospace label — and the block texts are the editors' own, in every language the editors offer: German (language: "de", the default) and English read off the running editors, the other 34 — "fr", "es", "ja", "zh-cn", ... — from the translation service the editors load at run time (approved strings only; an untranslated block keeps its English text, as in the editor). A bare code picks the regional variant the editor ships ("es" → es-ES). The end and else markers follow the language (fin/sinon, □□□/□□□□□, ...); `ende`, `end` and `else` work everywhere.

```
#makecode("
beim Start
  zeige Symbol [Herz v]
ende
wenn Knopf [A v] geklickt
  zeige Zahl ((1) + (2))
  pausiere (ms) (100)
ende
")
```

A unit the editor shows in parentheses is typed like a value and drawn as the label (`pausiere (ms) (100)`). The LED matrix takes 25 cells, `#` lit and `.` dark (`zeige LEDs [#...#|.#.#.|..#..|.#.#.|#...#]`), the melody editor eight notes or rests (`spiele (Melodie [C D E F - - - -] mit Tempo (120) (bpm)) [bis zum Ende v]`). `ende` closes a C-block, `ansonsten` opens the else branch with the editor's `-` and `+` buttons. `raw-makecode()` renders ````makecode`, ````microbit` and ````calliope` fences; `makecode-parse()` returns the AST. `set-blockst(profile: "makecode-calliope")` sets the default profile for `makecode()`, `colors:` overrides single categories, and the themes (`grayscale`, `high-contrast`, `print`) and line numbers work as for Scratch. Unknown labels are drawn as written with the category from a `::kategorie` suffix (`basic`, `input`, `music`, `led`, `radio`, `loops`, `logic`, `variables`, `math`, `functions`, `arrays`, `text`, `game`, `images`, `pins`, `serial`, `control`).



6 Labels and Line Numbers

blockst provides a label system for creating line-aware worksheets. Lines ending with `#label-name` are tagged and can be referenced later.

6.1 `scratch-labels()` — Extract Labels

Parses Scratch text and returns a dictionary mapping label names to line numbers.

```
#let labels = scratch-labels("
repeat (4) #loop
  move (20) steps #step
  turn cw (90) degrees
end
")
// labels = {"loop": 1, "step": 2}
```

6.2 `blockst-labels()` — Query Labels

Queries globally collected labels from all rendered `scratch()` calls.

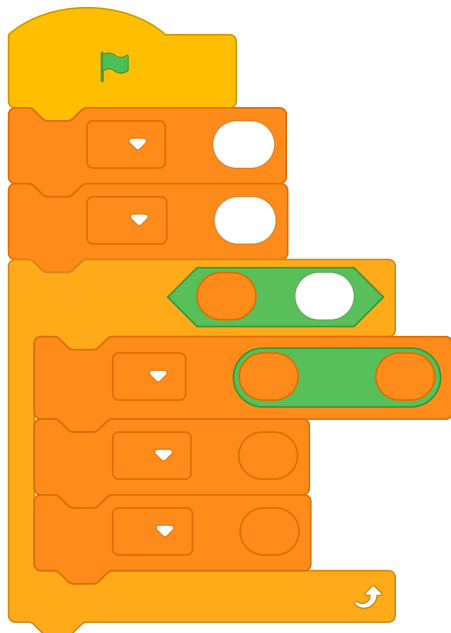
```
#blockst-labels("loop") // → line number or "NaN"
#blockst-labels()       // → full label → line dictionary
```

6.3 `blockst-register-labels()` — Pre-register Labels

Register labels globally without rendering blocks. Useful when labels are needed before the first block output.

```
#blockst-register-labels("
repeat (4) #loop
  move (20) steps #step
end
")
```

Euclidean Algorithm (gcd label walkthrough)



Explanation of labeled lines

- **Line 4:** Loop condition $b \neq 0$ controls termination.
- **Line 5:** Core rule: $r = a \bmod b$.
- **Line 7:** State update that advances to the next pair.

7 @category — Quick Color Defaults

Use `@category` prefix to force a block's category color, even without matching full localized syntax.

```
@motion           // → default motion block
@motion free text // → unrecognized block in motion color
```

Available categories:

Token	Normalized	Default Block
@motion	motion	move (10) steps
@looks	looks	say [Hello!]
@sound	sound	play sound [pop v]
@events	events	when green flag clicked
@control	control	repeat (10)
@sensing	sensing	ask [What's your name?] and wait
@operators	operators	(() + ())
@variable	variables	set [var v] to (0)
@list	lists	add (thing) to [list v]
@pen	pen	clear
@event	events	(alias for events)
@operator	operators	(alias for operators)

When `@category` is followed by text that matches a known block in that category, the actual block is rendered:

```
@list add (12) to [my list v] // → DATA_ADDTOLIST
@variable change [score v] by (1) // → DATA_CHANGEVARIABLEBY
```

If the text does not match any known block, the fallback is an `unrecognized` block in the forced category color.

8 Parsing API

8.1 `scratch-parse()` — Parse to AST

Parses Scratch text to an abstract syntax tree for programmatic use.

```
#scratch-parse(  
  text,           // Scratch block text  
  language: "en", // locale for parsing  
)
```

Returns a nested structure representing blocks, inputs, and bodies.

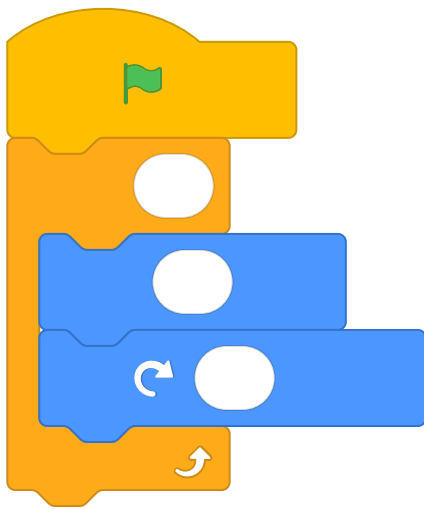
9 Markdown Code Blocks with raw-scratch

The `raw-scratch()` show rule converts scratch code fences into rendered blocks automatically.

```
#show: raw-scratch(language: "en") // locale for block text
```

Then use scratch code fences in your document:

```
when green flag clicked
repeat (4)
  move (30) steps
  turn cw (90) degrees
end
```



Optionally pass language: `raw-scratch(language: „de“)` in the show rule.

10 Theme and Styling Examples

```
#let script = "when green flag clicked
go to (random position v)
turn cw (30) degrees"

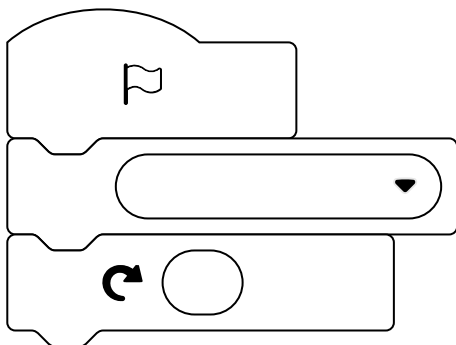
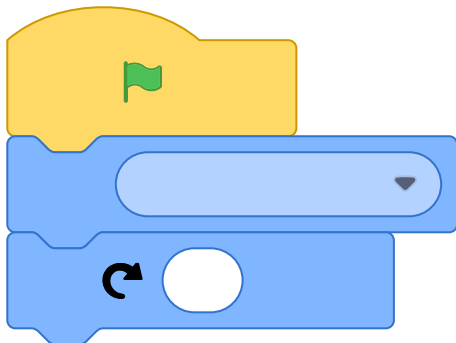
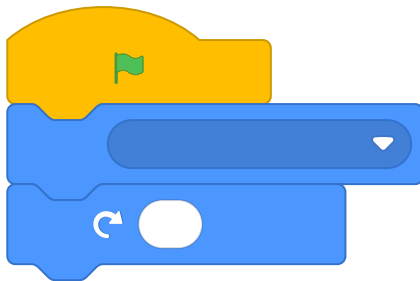
#blockst(inset-scale: 50%)[#scratch(script)]

#v(5mm)

#blockst(theme: "high-contrast")[#scratch(script)]

#v(5mm)

#blockst(theme: "print")[#scratch(script)]
```



11 Turtle Graphics / Executable Scratch

blockst includes an execution engine that runs Scratch programs visually — ideal for demonstrating program flow and pen drawing.

11.1 scratch-run Module

Import via the `scratch-run` module:

```
#import "@preview/blockst:0.4.0": scratch-run, set-scratch-run

#scratch-run.stage("...", scale: 2)
#scratch-run.grid("...", grid: true)
```

11.1.1 scratch-run.stage() — Stage Canvas

Renders pen drawing on a stage canvas, like the Scratch stage.

```
#scratch-run.stage(
  program,
  language: "en",      // locale for block text
  size: auto,          // (width, height) in pixels (default 480×360)
  scale: auto,         // drawing scale multiplier
  start: auto,         // (x: 0, y: 0, angle: 90)
  pen: auto,           // (down: false, color: ..., size: ...)
  background: auto,    // background color (e.g. white, black)
  cursor: auto,        // show/hide turtle cursor (default: true)
  border: auto,        // show/hide stage border (default: true)
)
```

11.1.2 scratch-run.grid() — Coordinate Grid

Renders the drawing on a Cartesian coordinate grid with optional axes and grid lines.

```
#scratch-run.grid(
  program,
  language: "en",      // locale for block text
  x: auto,             // view bounds (tuple, e.g. (-10, 10))
  y: auto,             // view bounds (tuple)
  step: auto,          // grid step size
  scale: auto,         // drawing scale multiplier
  start: auto,         // (x: 0, y: 0, angle: 90)
  pen: auto,           // (down: false, color: ..., size: ...)
  background: auto,    // background color
  axes: auto,          // show axis lines (default: false)
  grid: auto,          // show grid lines (default: false)
  grid-style: auto,    // grid line stroke (default: 0.5pt + gray)
  cursor: auto,        // show/hide turtle cursor (default: true)
  fit: auto,           // auto-fit view to drawing bounds
)
```

11.1.3 set-scratch-run() — Run Global Defaults

```
#set-scratch-run(
  scale: none,         // default scale for all runs
  start: none,         // (x: 0, y: 0, angle: 90)
  pen: none,           // (down: false, color: ..., size: ...)
  background: none,    // default background color
)
```

```

cursor: none,      // show/hide turtle cursor (default: true)
stage: none,       // (size: (480, 360), border: true)
grid: none,        // (visible: false, axes: false, step: auto, style: auto)
)

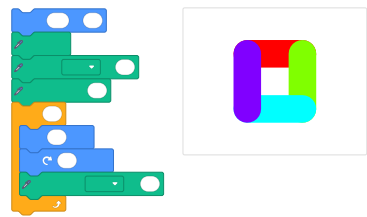
```

11.1.4 Supported Execution Commands

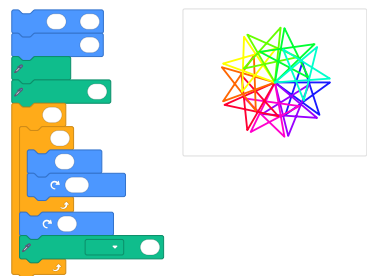
Category	Commands
Motion	move, turn-right, turn-left, set-direction, go-to, set-x, set-y, change-x, change-y
Pen	pen-down, pen-up, set-pen-color, set-pen-size, change-pen-size, erase-all, stamp, set-pen-param, change-pen-param
Variables	set-variable, change-variable, variable
Operators	plus, minus, multiply, divide, modulo, random, round, greater, less, equals, op-and, op-or, op-not
Control	repeat, repeat-until, if-then, if-else, wait
Looks	say, think
Shapes	square, triangle, circle, star, spiral
German	gehe, drehe-rechts, drehe-links, setze-richtung, gehe-zu, stift-ein, stift-aus, setze-stiftfarbe-auf, setze-stiftdicke, setze-variable, aendere-variable, groesser, kleiner, gleich, und, oder, nicht, mal, geteilt, zufallszahl

11.1.5 Complete Example

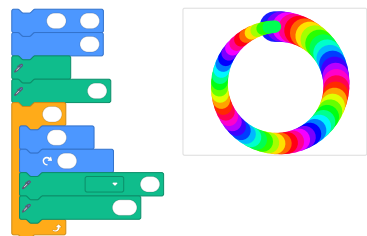
1) Hue Square



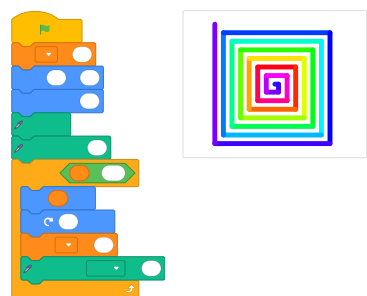
2) Star Rosette (Nested repeat)



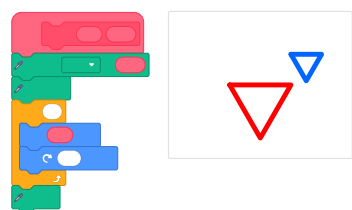
3) Colored Spiral (Hue + pen size)



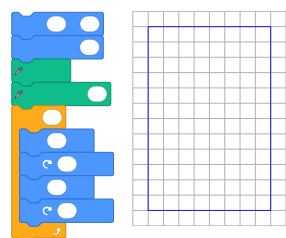
4) Conditional Loop



5) Custom Block: Triangle Pattern



6) Grid Preview (Axes + fixed bounds)



```
#let square-program = "  
go to x: (-45) y: (45)  
pen down  
set pen [color v] to (0)  
set pen size to (45)  
repeat (4)  
  move (90) steps  
  turn cw (90) degrees  
  change pen [color v] by (25)  
end"  
  
#set-scratch-run(  
  stage: (size: (300, 240)),  
  start: (x: 0, y: 0, angle: 90),  
)  
  
#grid(  
  columns: (auto, auto),  
  gutter: 6mm,  
  [#scratch(square-program)],  
  [#scratch-run.stage(square-program, scale: 2)],  
)
```

12 SB3 Import API

blockst can read real Scratch 3 project files (`.sb3`) and extract scripts, variables, lists, images, and screen previews.

12.1 Basic Workflow

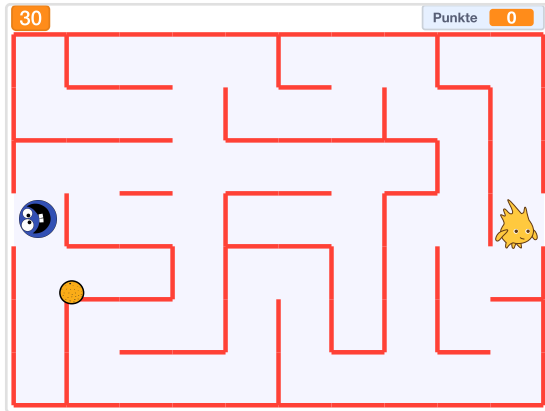
```
#let project = read("my-project.sb3", encoding: none)

#sb3.render-sb3-scripts(project, language: "en", target: "Sprite1")
```

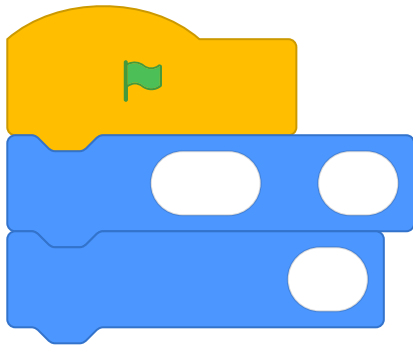
The `.sb3` file must be read as raw bytes (`encoding: none`).

12.2 `render-sb3-scripts()`

```
#sb3.render-sb3-scripts(
  sb3-bytes,           // raw .sb3 file bytes (read with encoding: none)
  script-number: auto, // global script index (1-based)
  target-script-number: auto, // script index within selected target
  target: auto,        // filter by target name ("Stage", sprite name, or auto for
all)
  sb3-plugin: auto,     // WASM plugin path (auto = bundled plugin)
  language: "en",       // locale for block text
  show-headers: auto,   // show target name header
  header-gap: 1.5mm,    // spacing between target name and scripts
  script-gap: 3mm,      // spacing between individual scripts
)
```



2. Sprite: Pacman - Script 2



12.3 render-sb3-variables()

```
#sb3.render-sb3-variables(
  sb3-bytes,           // raw .sb3 file bytes
  target: auto,        // filter by target name
  target-variable-name: auto, // filter by variable name
  target-variable-number: auto, // variable index within target (1-based)
  sb3-plugin: auto,    // WASM plugin path
  language: "en",      // locale for display text
  show-target-headers: auto, // show target name header
  target-gap: 2mm,     // spacing between targets
  item-gap: 0.8mm,     // spacing between variable items
)
```

12.4 render-sb3-lists()

```
#sb3.render-sb3-lists(
  sb3-bytes,           // raw .sb3 file bytes
  target: auto,        // filter by target name
  target-list-name: auto, // filter by list name
  target-list-number: auto, // list index within target (1-based)
  sb3-plugin: auto,    // WASM plugin path
  language: "en",      // locale for display text
  show-target-headers: auto, // show target name header
  target-gap: 2mm,     // spacing between targets
  item-gap: 0.8mm,     // spacing between list items
)
```

12.5 Standalone Monitors

12.5.1 list-monitor()

```
#list-monitor(
  name: "List",          // display name shown in header
  items: (),             // array of values to display
  width: 5.2cm,          // monitor width
  height: auto,          // auto-grows to fit content
  length-label: auto,    // show length indicator (e.g. "length: 3")
)
```

12.5.2 variable-monitor()

```
#variable-monitor(
  name: "Variable",     // display name shown in header
  value: 0,              // current value to display
)
```

Counter 30

Punkte 0

Highscore	
1	Thomas
2	Alex
3	Lukas
+ Length: 3 =	

12.6 screen-preview()

```
#sb3.sb3-screen-preview(
  sb3-bytes,            // raw .sb3 file bytes
  width: 480,           // stage rendering width in pixels
  height: 360,          // stage rendering height in pixels
  unit: 1,              // size multiplier (2 = double size)
  background: none,     // override background color
  show-border: true,    // show stage border
  show-backdrop: true,  // show costume/backdrop image
  monitor-scale: 1.5,   // scale factor for variable/list overlays
  language: auto,       // locale for monitor text
)
```

Renders a static Scratch stage preview with sprites, backdrop, and monitors.

12.7 Image Helpers

```
// List all image assets in the project
#sb3.sb3-image-assets-catalog(sb3-bytes, target: auto)

// Render a single image asset
#sb3.sb3-image(
  sb3-bytes,            // raw .sb3 file bytes
```

```
target: auto,          // filter by sprite/stage name
image-number: auto,    // global image index (1-based)
target-image-number: auto, // image index within target (1-based)
image-name: auto,      // filter by image name (e.g. "costume1")
width: auto,           // output width (auto = original size)
height: auto,          // output height
)
```

12.8 Catalog Helpers

```
// Grouped script metadata (targets, scripts, blocks count)
#sb3.sb3-scripts-catalog(sb3-bytes)

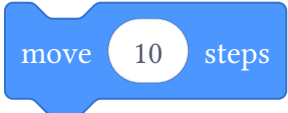
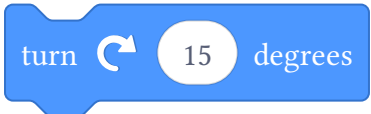
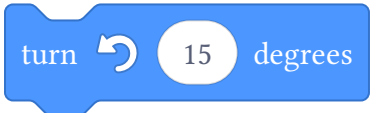
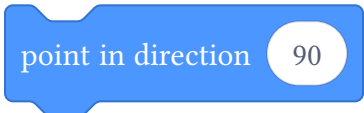
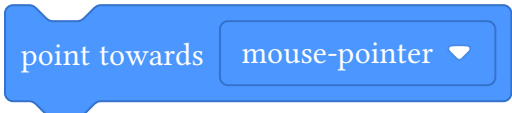
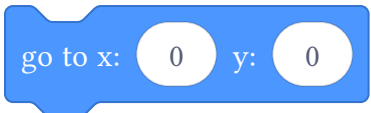
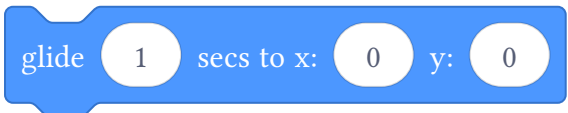
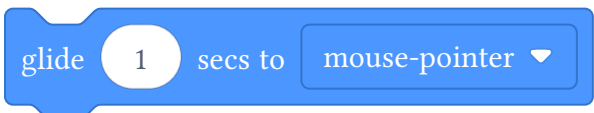
// Target states (variables, lists, and sprite properties)
#sb3.sb3-state-catalog(sb3-bytes)

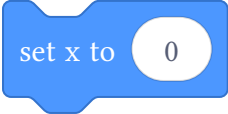
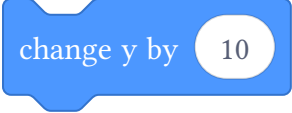
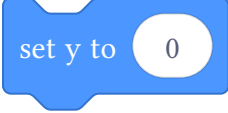
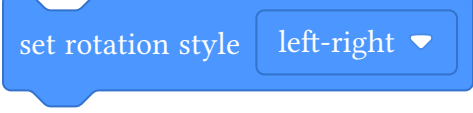
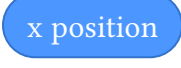
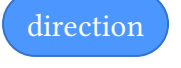
// Convert a specific SB3 script to Scratch text
#sb3.sb3-bytes-to-scratch-text(
  sb3-bytes,          // raw .sb3 file bytes
  script-number: auto, // global script index (1-based)
  language: "en",     // locale for output text
)
```

13 Catalog

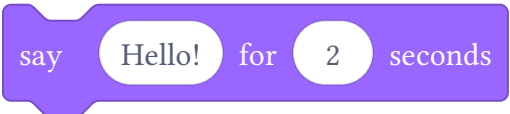
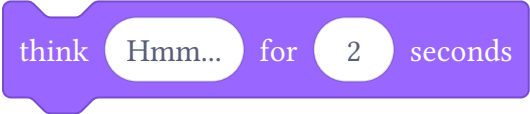
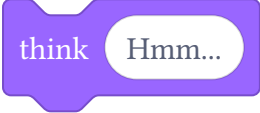
The catalog shows every block in each Scratch 3 category, rendered live.

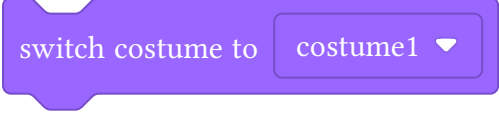
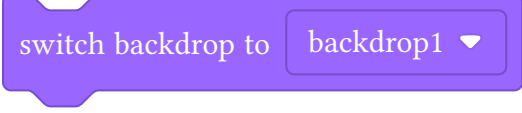
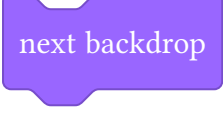
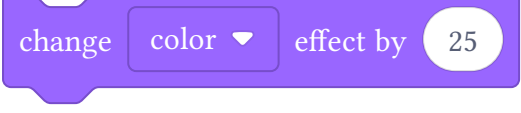
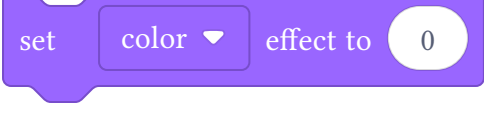
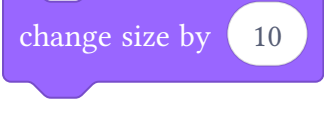
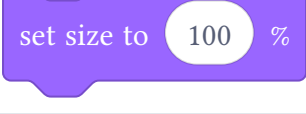
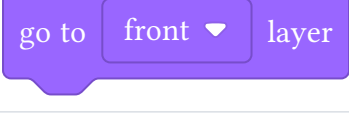
13.1 Motion

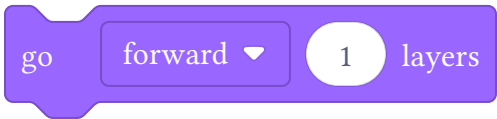
Block	Code
	<code>move (10) steps</code>
	<code>turn cw (15) degrees</code>
	<code>turn ccw (15) degrees</code>
	<code>point in direction (90)</code>
	<code>point towards [mouse-pointer v]</code>
	<code>go to x: (0) y: (0)</code>
	<code>go to [mouse-pointer v]</code>
	<code>glide (1) secs to x: (0) y: (0)</code>
	<code>glide (1) secs to [mouse-pointer v]</code>
	<code>change x by (10)</code>

Block	Code
	<code>set x to (0)</code>
	<code>change y by (10)</code>
	<code>set y to (0)</code>
	<code>if on edge, bounce</code>
	<code>set rotation style [left-right v]</code>
	<code>(x position)</code>
	<code>(y position)</code>
	<code>(direction)</code>

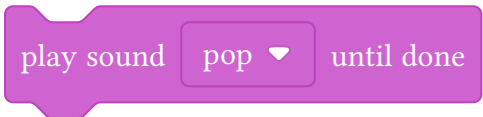
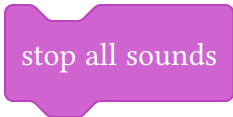
13.2 Looks

Block	Code
	<code>say [Hello!] for (2) seconds</code>
	<code>say [Hello!]</code>
	<code>think [Hmm...] for (2) seconds</code>
	<code>think [Hmm...]</code>

Block	Code
	<code>show</code>
	<code>hide</code>
	<code>switch costume to [costume1 v]</code>
	<code>next costume</code>
	<code>switch backdrop to [backdrop1 v]</code>
	<code>next backdrop</code>
	<code>change [color v] effect by (25)</code>
	<code>set [color v] effect to (0)</code>
	<code>clear graphic effects</code>
	<code>change size by (10)</code>
	<code>set size to (100) %</code>
	<code>go to [front v] layer</code>

Block	Code
	<code>go [forward v] (1) layers</code>
	<code>(costume [number v])</code>
	<code>(backdrop [number v])</code>
	<code>(size)</code>

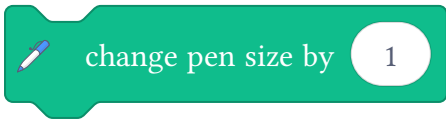
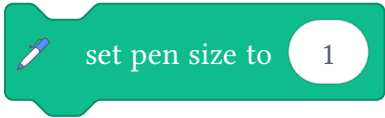
13.3 Sound

Block	Code
	<code>play sound [pop v] until done</code>
	<code>start sound [pop v]</code>
	<code>stop all sounds</code>
	<code>change [pitch v] effect by (10)</code>
	<code>set [pitch v] effect to (100)</code>
	<code>clear sound effects</code>
	<code>change volume by (-10)</code>

Block	Code
	<code>set volume to (100) %</code>
	<code>(volume)</code>

13.4 Pen

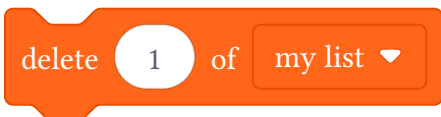
Block	Code
	<code>erase all</code>
	<code>stamp</code>
	<code>pen down</code>
	<code>pen up</code>
	<code>set pen color to [#ff0000]</code>
	<code>change pen color by (10)</code>
	<code>set pen color to (50)</code>
	<code>change pen shade by (10)</code>
	<code>set pen shade to (50)</code>

Block	Code
	<code>change pen size by (1)</code>
	<code>set pen size to (1)</code>

13.5 Variables

Block	Code
	<code>set [my variable v] to (0)</code>
	<code>change [my variable v] by (1)</code>
	<code>show variable [my variable v]</code>
	<code>hide variable [my variable v]</code>
	<code>(my variable)</code>

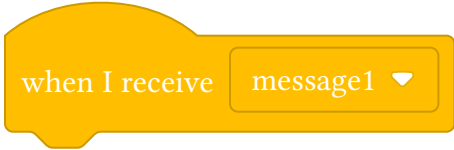
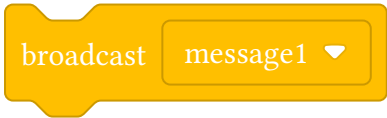
13.6 Lists

Block	Code
	<code>add [thing] to [my list v]</code>
	<code>delete (1) of [my list v]</code>
	<code>delete all of [my list v]</code>

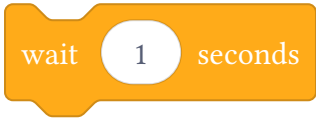
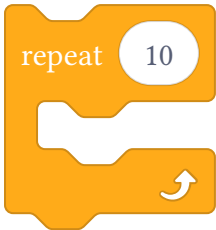
Block	Code
	<code>insert [thing] at (1) of [my list v]</code>
	<code>replace item (1) of [my list v] with [thing]</code>
	<code>(item (1) of [my list v])</code>
	<code>(item # of [thing] in [my list v])</code>
	<code>(length of [my list v])</code>
	<code><[my list v] contains [thing]?></code>
	<code>show list [my list v]</code>
	<code>hide list [my list v]</code>

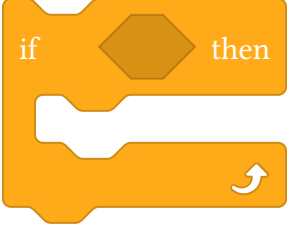
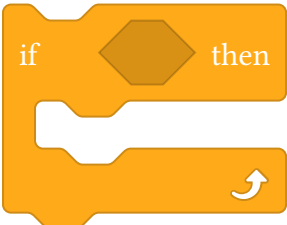
13.7 Events

Block	Code
	<code>when green flag clicked</code>
	<code>when [space v] key pressed</code>
	<code>when this sprite clicked</code>

Block	Code
	<code>when backdrop switches to [backdrop1 v]</code>
	<code>when [loudness v] > (10)</code>
	<code>when I receive [message1 v]</code>
	<code>broadcast [message1 v]</code>
	<code>broadcast [message1 v] and wait</code>

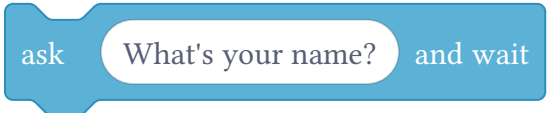
13.8 Control

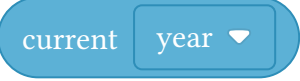
Block	Code
	<code>wait (1) seconds</code>
	<code>repeat (10)</code>
	<code>end</code>
	<code>forever</code>

Block	Code
	<code>end</code>
	<code>if <> then</code>
	<code>end</code>
	<code>if <> then</code>
	<code>else</code>
	<code>end</code>
	<code>wait until <></code>
	<code>repeat until <></code>
	<code>end</code>
	<code>stop [all v]</code>

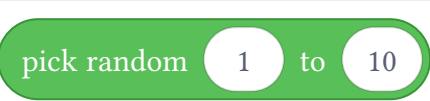
Block	Code
	<code>when I start as a clone</code>
	<code>create clone of [myself v]</code>
	<code>delete this clone</code>

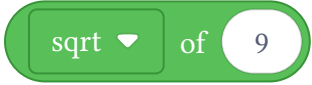
13.9 Sensing

Block	Code
	<code><touching [mouse-pointer v]?></code>
	<code><touching color [#ff0000]?></code>
	<code><color [#ff0000] is touching [#00ff00]?></code>
	<code>(distance to [mouse-pointer v])</code>
	<code>ask [What's your name?] and wait</code>
	<code>(answer)</code>
	<code><key [space v] pressed?></code>
	<code><mouse down?></code>
	<code>(mouse x)</code>
	<code>(mouse y)</code>

Block	Code
	<code>(loudness)</code>
	<code>(timer)</code>
	<code>reset timer</code>
	<code>((x position v) of [Sprite1 v])</code>
	<code>(current [year v])</code>
	<code>(days since 2000)</code>
	<code>(username)</code>

13.10 Operators

Block	Code
	<code>(() + ())</code>
	<code>(() - ())</code>
	<code>(() * ())</code>
	<code>(() / ())</code>
	<code>(pick random (1) to (10))</code>
	<code><() > ()></code>
	<code><() < ()></code>

Block	Code
	<code><() = ()></code>
	<code><> and <></code>
	<code><> or <></code>
	<code>not <></code>
	<code>(join [apple] [banana])</code>
	<code>(letter (1) of [apple])</code>
	<code>(length of [apple])</code>
	<code>([apple] contains [a]?)</code>
	<code>(() mod ())</code>
	<code>(round ())</code>
	<code>([sqrt v] of (9))</code>

14 Contributing

Contributions are welcome: bug reports, missing blocks, parser improvements, rendering polish, docs, and new localizations.

- **Repository:** github.com/Loewe1000/blockst
- **License:** MIT

15 API Reference

The reference is generated from the `///` comments in the source. It covers the public rendering API in `libs/scratch/api.typ`, which `lib.typ` and `package.typ` re-export unchanged.

15.1 blockly

```
blockly(
  text,
  profile: auto,
  language: auto,
  theme: auto,
  scale: auto,
  font: auto,
  line-numbering: auto,
  line-numbers: auto,
  line-number-start: auto,
  line-number-first-block: auto,
  line-number-gutter: auto,
  inset-scale: auto,
  colors: auto,
)
```

Render Blockly blocks from text. Same notation as `scratch()`, drawn with Blockly's shapes — notch, puzzle tab, boxed fields — and the vocabulary, categories and colours of a profile:

- `"blockly"` (the default): today's flat look, Blockly's German wording
- `"blockly-klassisch"`: the pre-2019 look
- `"jwinf"`: jwinf.de — classic geometry, the robot and turtle world blocks, and the palette of the robot training tasks
- `"jwinf-turtle"`: the same with the colours of the Freie Turtle-Umgebung

`colors: (logik: "#73cc47")` overrides single categories of any profile.

Labels that no profile knows are drawn as written, with the category taken from a `::kategorie` suffix — on jwinf the normal case, because the same block reads differently from task to task.

```
#blockly("wiederhole (4) mal:\n gehe nach rechts\nende", profile: "jwinf")
```

Name	Vorgabe
text	—
profile	auto
language	auto
theme	auto
scale	auto
font	auto
line-numbering	auto
line-numbers	auto
line-number-start	auto
line-number-first-block	auto
line-number-gutter	auto
inset-scale	auto

Name	Vorgabe
colors	auto

15.2 blockly-parse

```
blockly-parse(text, language: "de", profile: "blockly")
```

Parse Blockly text to AST (for programmatic use).

Name	Vorgabe
text	–
language	"de"
profile	"blockly"

15.3 blockst

```
blockst(
  theme: auto,
  scale: auto,
  font: auto,
  line-numbering: auto,
  line-numbers: auto,
  line-number-start: auto,
  line-number-first-block: auto,
  line-number-gutter: auto,
  inset-scale: auto,
  language: auto,
  colors: auto,
  spacing: 1.5em,
  body,
)
```

Optional container for grouped blocks with theme/scale override. Only needed when a specific group should differ from global settings.

Name	Vorgabe
theme	auto
scale	auto
font	auto
line-numbering	auto
line-numbers	auto
line-number-start	auto
line-number-first-block	auto
line-number-gutter	auto
inset-scale	auto
language	auto
colors	auto
spacing	1.5em
body	–

15.4 blockst-labels

```
blockst-labels(name)
```

Read globally collected line labels from all previously rendered `scratch()` blocks. With a name: `#blockst-labels("start", default: "?")`. Without a name: returns the full label-to-line dictionary.

15.5 blockst-register-labels

```
blockst-register-labels(text, language: "en")
```

Register labels globally without rendering blocks. Useful when labels are needed before the first `#scratch()` output appears.

Name	Vorgabe
text	–
language	"en"

15.6 makecode

```
makecode(
  text,
  profile: auto,
  language: auto,
  theme: auto,
  scale: auto,
  font: auto,
  line-numbering: auto,
  line-numbers: auto,
  line-number-start: auto,
  line-number-first-block: auto,
  line-number-gutter: auto,
  inset-scale: auto,
  colors: auto,
)
```

Render MakeCode blocks from text — the micro:bit and Calliope mini editors' look: Blockly's `zelos` renderer with MakeCode's monospace labels. Same notation as `scratch()`; `(...)` is a value, `[... v]` a dropdown, `<...>` a boolean, `ende` / `end` closes a C-block and `sonst` / `else` opens the else branch. The profile picks the target: `"makecode"` (micro:bit, the default) or `"makecode-calliope"`; the language is any of the editors' 36 (`"de"` by default, `"en"`, `"fr"`, `"es"`, `"zh-cn"`, ...), with the block texts the editors show in that language. The end marker follows the language (`ende`, `end`, `fin`, ...); `end` and `ende` work in every language.

```
#makecode("beim Start\n zeige Zahl (0)\nende")
```

Name	Vorgabe
text	–
profile	auto
language	auto
theme	auto
scale	auto
font	auto
line-numbering	auto
line-numbers	auto

Name	Vorgabe
line-number-start	auto
line-number-first-block	auto
line-number-gutter	auto
inset-scale	auto
colors	auto

15.7 makecode-parse

```
makecode-parse(text, language: "de", profile: "makecode")
```

Parse MakeCode text to AST (for programmatic use).

Name	Vorgabe
text	—
language	"de"
profile	"makecode"

15.8 raw-blockly

```
raw-blockly(..args)
```

Enable Blockly code blocks in raw text:

```
#show: raw-blockly()
```blockly
wiederhole (4) mal:
ende
```
```

A `jwinf` fence uses the `jwinf` profile whatever the arguments say, a `jwinf-turtle` fence the turtle sandbox's colours.

15.9 raw-makecode

```
raw-makecode(..args)
```

Enable MakeCode code blocks in raw text:

```
#show: raw-makecode()
```makecode
beim Start
 zeige Zahl (0)
ende
```
```

A `microbit` fence is the same; a `calliope` fence uses the Calliope mini profile whatever the arguments say.

15.10 raw-scratch

```
raw-scratch(..args)
```

Enable scratch code blocks in raw text:

```
#show: raw-scratch()
```

With language: `#show: raw-scratch(language: "de")`.

15.11 scratch

```
scratch(
  text,
  language: auto,
  theme: auto,
  scale: auto,
  font: auto,
  line-numbering: auto,
  line-numbers: auto,
  line-number-start: auto,
  line-number-first-block: auto,
  line-number-gutter: auto,
  inset-scale: auto,
  colors: auto,
)
```

Render scratch blocks from text. Works standalone or inside `#blockst[...]`. Supports 26 languages: en, de, fr, es, it, pt, nl, pl, ru, ja, ...

| Name | Vorgabe |
|-------------------------|---------|
| text | – |
| language | auto |
| theme | auto |
| scale | auto |
| font | auto |
| line-numbering | auto |
| line-numbers | auto |
| line-number-start | auto |
| line-number-first-block | auto |
| line-number-gutter | auto |
| inset-scale | auto |
| colors | auto |

15.12 scratch-execute

```
scratch-execute(text, language: "en")
```

Execute scratch text, producing scratch-run commands.

| Name | Vorgabe |
|----------|---------|
| text | – |
| language | "en" |

15.13 scratch-labels

```
scratch-labels(text, language: "en")
```

Parse scratch text and return a dictionary mapping `#labels` to rendered line numbers.

| Name | Vorgabe |
|------|---------|
| text | – |

| Name | Vorgabe |
|----------|---------|
| language | "en" |

15.14 scratch-parse

```
scratch-parse(text, language: "en")
```

Parse scratch text to AST (for programmatic use).

| Name | Vorgabe |
|----------|---------|
| text | – |
| language | "en" |

15.15 set-blockst

```
set-blockst(
  theme: none,
  profile: none,
  colors: none,
  scale: none,
  stroke-width: none,
  font: none,
  line-numbering: none,
  line-numbers: none,
  line-number-start: none,
  line-number-first-block: none,
  line-number-gutter: none,
  inset-scale: none,
  language: none,
)
```

Global settings applied to all scratch() and sb3 calls.

colors lays the document's own category colours over the profile's palette: set-blockst(colors: (logik: "#73cc47")) — a hex string or a Typst colour per category. The shades a theme derives (bevel, stroke, high-contrast, grayscale) follow the new fill.

| Name | Vorgabe |
|-------------------------|---------|
| theme | none |
| profile | none |
| colors | none |
| scale | none |
| stroke-width | none |
| font | none |
| line-numbering | none |
| line-numbers | none |
| line-number-start | none |
| line-number-first-block | none |
| line-number-gutter | none |
| inset-scale | none |
| language | none |